

The Sovereign Substrate

Reference Architecture and Implementation

S. Wheeler

v1.0

Companion to *The Sovereign Substrate: A constitutional architecture for governed computation* (S. Wheeler, v1.0, 2026).

Correspondence: swheeler-research@proton.me

Paper repository: <https://github.com/swheeler-research/substrate-research>

Reference implementation: <https://github.com/swheeler-research/substrate-reference>

Licence: CC BY 4.0

Contents

Abstract	1
1. Background and scope	1
2. The three primitives	2
2.1 Functional units	2
2.2 State units	3
2.3 Credential units	3
2.4 Compositional uniformity	4
3. The eight mechanisms	4
3.1 Compile-at-commit	4
3.1.1 Wilful inclusion check	5
3.1.2 Authority chain resolution	5
3.1.3 Policy collection	5
3.1.4 Structured precondition collection and joint satisfiability	5
3.1.5 Calibration-handling validation	6
3.1.6 Compiled-form emission and witnessing	6
3.1.7 What compile-at-commit does not do	6
3.2 Roll-up under strictest-binding-wins	7
What roll-up does not do	8
3.3 Wilful inclusion	8
What wilful inclusion does not do	8
3.4 Refusal under non-reconcilable composition	8
What refusal does not do	9
3.5 Runtime evaluation against the compiled form	10
What runtime evaluation does not do	11
3.6 The uniform invalidation surface	11
What the uniform invalidation surface does not do	12
3.7 Administrative acts on the ledger	12
What administrative acts do not do	13
3.8 Federation of archives under multi-custodian quorum	13
Adversarial and failure assumptions	14
What federation does not do	15
4. The reference implementation	15
4.1 Module correspondence to the specification	16
4.2 What the prototype does not yet substantiate	17
4.3 Conformance and interoperability	18
4.4 What the prototype demonstrates about cost	18
5. The architecture under exercise	20
5.1 Robodebt	20

5.2 Boeing 737 MAX	21
5.3 London Whale	23
5.3.1 Routine position under the original VaR	23
5.3.2 Recalibration	24
5.3.3 Drift	24
5.3.4 Over-limit position refused	24
5.3.5 Escalation	24
5.3.6 Backtest after drift reset	24
5.3.7 OCC audit	25
6. What the architecture does not guarantee	26
7. Closing	29

Abstract

Modern computational systems lack an independent governance substrate between institutional policy and executable behaviour. Authority chains are reconstructed retrospectively from logs the operating institution controls; policy is enforced by the institution whose behaviour it is meant to constrain; auditability depends on institutional cooperation. The companion monograph develops this structural fault, situates it against twelve consequential cases, and argues for an architectural response.

This document specifies the architecture at reference depth and documents the reference implementation that conforms to it. Three primitive types (functional, state, and credential units) compose under eight protocol mechanisms (compile-at-commit, roll-up under strictest-binding-wins, wilful inclusion, refusal under non-reconcilable composition, runtime evaluation against the compiled form, the uniform invalidation surface, administrative acts on the ledger, and federation of archives under multi-custodian quorum). The mechanisms compose uniformly across scales: the same vocabulary describes governance at a single functional unit, at an institutional substrate, and at a cooperative substrate of sovereigns.

The reference implementation comprises roughly three and a half thousand lines of core substrate code, with four and a half thousand lines of tests and nine thousand lines of worked examples; it exercises 225 tests and provides twelve end-to-end demonstrations against substantively different problems. Six commitments the monograph specifies are named in this document as not yet substantiated by the prototype: extraction-maturity level as structured unit content, observability provenance as structured unit content, visibility class for policy content, twinning of constitutional source credentials with guardian-quorum revocation, persistent drift state surviving prototype restart, and calibration-handling contract as first-class field on the compiled form. The architectural commitments are stable; what differs is the engineering depth at which the prototype expresses them.

The architecture does not solve politics, does not prevent corruption, does not prevent coercive sovereigns, and does not guarantee fairness. What it does is make authority preservation through computational composition a structural property of the system, rather than a procedural commitment external to it.

1. Background and scope

The companion monograph, *The Sovereign Substrate: A constitutional architecture for governed computation*, is the authoritative architectural treatise. It develops the structural problem the architecture addresses, situates the work against twelve canonical cases, and specifies the architecture's commitments at the depth the constitutional argument requires. This document is the companion at reference depth, written for the systems engineer, the security researcher, the standards-body technical staff member, or the conforming-implementer who needs the architecture without the surrounding constitutional argument. The constitutional rationale lives in the monograph; this document does not re-make it.

What this document specifies: the three primitive types and their internal typings, the eight protocol mechanisms at the depth required to write a conforming implementation, the reference implementation's correspondence to the specification, and the bounds of the architecture's claims, named where each claim is specified rather than relegated to a separate section. What this document does not specify: a wire protocol. The substrate is at the architectural layer; wire-protocol specification is downstream work, conceivable as future IETF or W3C standards-track activity building on RFC 9162 for witnessing, RFC 5280 for credential structure, the W3C Verifiable Credentials data model for credential expression, and Sigstore specifications for transparency logs. The architecture is what this document specifies; the protocol is what conforming implementations satisfy at the level of mechanical commitments; the substrate is what an operator runs. These distinctions are preserved throughout.

The cases this document exercises against are three of the monograph's twelve, selected for mechanism coverage: Robodebt (the runtime-policy distinction), Boeing 737 MAX (certification as a substrate pattern), and London Whale (the densest single composition of mechanisms in the case set). The other nine cases are exercised in the reference implementation's `examples/` directory and developed at substantive depth in the monograph. The selection reflects this document's job: to specify the architecture and show it running, not to argue for its consequence.

2. The three primitives

The architecture admits three primitive types. The set is closed: every unit is one of the three, and no fourth is introduced. The primitives' internal structure is uniform: each is a content-and-references object, identified by the content-addressable hash of its canonicalised content. The three differ in what they carry and how they compose.

2.1 Functional units

A functional unit is the primitive for the work of producing. Its content declares the unit's purpose, its input and output specifications, its preconditions, and the contract pattern under which its behaviour is characterised. Its references identify the credentials under which the unit operates, the policies that bind it, and the sub-units its implementation may invoke.

Three contract patterns are admitted. A specification-bounded unit declares its behaviour through formal input and output types and is conformant by virtue of meeting the specification; verification reduces to type-checking the implementation against the specification. A behaviour-characterised unit declares acceptance bands rather than exact behaviour; its content includes a calibration claim about the reliability of its outputs, a drift criterion stating the conditions under which the calibration claim is held to fail, and an output field through which the runtime observes calibration in operation. A hybrid unit composes specification-bounded and behaviour-characterised components within a single declaration; its content names which fields are bounded and which are characterised, and the runtime evaluates accordingly.

Preconditions on inputs are declared in the unit's content as structured constraints over the

input variables: numeric ranges, categorical sets, temporal windows. Preconditions enter compilation through the joint-satisfiability check in stage four of the compilation pipeline (§3.1.4); a unit that declares no preconditions contributes nothing to the check, which is the architectural point established in §5.1 below.

A functional unit's executable artefact is a content-addressed binary or interpretable source referenced from the unit's content, retrieved from the code archive at runtime. The executable artefact is not the unit; the unit is the content-and-references declaration of which the artefact is one component.

2.2 State units

A state unit is the primitive for what is held. Its content carries the data the unit represents; its references identify the credentials authorising the data and the policies governing access. The content is what the unit is at the moment of admission; references resolve through the credentials archive and the substrate's governance machinery.

Three mutability disciplines are admitted. An immutable state unit's content is fixed at admission; subsequent versions produce new content-addressable identities. A mutable state unit's content may be updated under the policies bound to its credentials, with each update producing a new content-addressed version; the unit's identity tracks the latest version and prior versions remain retrievable. An append-only state unit admits new content appended to its existing content but does not admit modification of prior content; this is the discipline ledgers and audit logs satisfy.

State units include the ledger itself, archive contents, and the compiled forms emitted by the compilation pipeline. A compiled form is an immutable state unit whose content records the resolved authority chain, the rolled-up policy stack, the reference resolutions, the calibration-handling validation, and the provenance attestation of the compilation. Compiled forms are placed in the code archive and registered on the operator's runtime; the runtime evaluates every act against the compiled form of its target unit.

2.3 Credential units

A credential unit is the primitive for the structures of recognition and permission. Its content declares the credential's purpose and constraints; its references identify the parent credentials under which the credential operates, the policies it carries, and the principals it recognises. A credential's authority chain is the directed acyclic graph rooted at constitutional source credentials and ending at the credential itself.

Three transfer disciplines are admitted. A bearer credential is transferable; its possession authorises its use, and revocation requires the operator to invalidate the credential's content identity. A delegated credential is bound to a specific principal under specific scope; its use requires the principal to present it under conditions the credential's content specifies. A capability credential is bound to a specific target unit or class of units; its use authorises action on the target and is otherwise inert.

Policy units are credential units operating in governance role. They are not a fourth primitive.

A policy unit's content declares the constraints it imposes; its references identify the units it binds and the authority under which it operates. The compilation pipeline collects policy units through the credential graph and composes them under the roll-up rule of §3.2.

2.4 Compositional uniformity

The vocabulary above is the entire primitive vocabulary the architecture admits. The same vocabulary describes a single functional unit on a personal substrate, an institutional substrate composing many units under cooperative-substrate witnessing, and a sovereign substrate operating at multi-national scale. There is no scale-specific primitive set, no separate vocabulary for institutional governance, no separate vocabulary for sovereign cooperation. An operator is whoever authors and runs a substrate; the three primitives compose the same way regardless of which operator runs them and at what scale.

This compositional uniformity is the architecture's central structural claim. It is not asserted; it is exhibited under exercise in §5 below. The architecture earns the claim by showing that the same vocabulary describes welfare adjudication (Robodebt), aircraft certification (Boeing 737 MAX), and financial-risk modelling under regulatory audit (London Whale) without bending or specialising. Where the prototype implements the vocabulary directly, the demonstrations run; where the monograph's six unimplemented commitments would otherwise apply, the demonstrations operate at the prototype's current engineering depth and the gaps are named in §4.

3. The eight mechanisms

The architecture's protocol-level commitments are eight mechanisms. Each is specified below at the depth required to write a conforming implementation. The first four (compile-at-commit, roll-up under strictest-binding-wins, wilful inclusion, refusal under non-reconcilable composition) describe what happens when a unit is committed to an operator's archive. The next two (runtime evaluation against the compiled form, the uniform invalidation surface) describe what happens at every subsequent act and at every event that invalidates a compiled form. The seventh (administrative acts on the ledger) describes the operator's interface to the substrate's own governance state. The eighth (federation of archives under multi-custodian quorum) describes how independent operators compose into cooperative substrates.

Each subsection ends with what the mechanism does not do. The limits are part of the specification, not deferred to a separate section.

3.1 Compile-at-commit

Compile-at-commit performs the structural work of unit admission once, at the moment the unit is added to the operator's code archive, and emits an immutable compiled form against which every subsequent invocation evaluates. Operational work at the act level is cheap; structural work is amortised across every subsequent invocation of the unit.

The pipeline has six stages. The reference implementation runs them in the following order.

3.1.1 Wilful inclusion check

The pipeline resolves the unit's transitive reference graph and verifies that every unit reachable through any chain of references appears in the source unit's own top-level reference set. A reference reachable only transitively, through an intermediate unit, fails the check. The wilful-inclusion rule prevents composition by hidden dependency: an operator authoring a unit must name every unit on which the unit transitively depends, and the compiler refuses units whose composition includes references the unit's content does not declare.

The check also verifies that the unit's reference set addresses every governance dimension the operator's authored content declares applicable to units of this type at this scale. Silence on any required dimension produces compilation refusal. The operator declares which dimensions are required; the compiler enforces that declaration; the unit's author cannot omit a dimension the operator's policy requires without the compiler refusing.

3.1.2 Authority chain resolution

The pipeline walks the unit's identity credential's parent credentials upward, then the parents of each in turn, until every chain terminates at one or more constitutional source credentials recognised within the operator's authored content. The traversal produces the unit's authority chain: a directed acyclic graph rooted at constitutional source credentials and ending at the unit's identity credential.

Authority chain resolution fails if any credential in the chain is revoked, expired, or unresolvable through the credentials archive; if any chain does not terminate at a constitutional source within the substrate's recognition scope; or if the constitutional source's anchoring natural persons are absent from the operator's authored content. The constitutional source is not a substrate-internal artefact; it is anchored to natural persons through institutional and legal apparatus the substrate does not itself provide.

3.1.3 Policy collection

The pipeline assembles the set of policy units in scope at this unit's compilation. The set includes the policies referenced by the unit's identity credential through its policy references, the policies referenced by every credential in the unit's authority chain through their policy references, the policies attached to the operator's substrate at the unit's scale through the operator's authored content, and the policies attached to each functional, state, and credential unit referenced by the unit, transitively through the references of those units' compiled forms.

The result is a set of functional units in governance role, each itself with an authority chain and a rolled-up policy stack already computed by its own prior compilation. Policies bind the unit by being collected here; the binding is structural, not configurational.

3.1.4 Structured precondition collection and joint satisfiability

The pipeline collects the structured preconditions declared across the source unit and the units in scope and checks their joint satisfiability per variable. For each input variable, the pipeline intersects the preconditions contributed by every unit that declares one. An empty intersection on any variable is a type mismatch; the compilation refuses, and the refusal names

the variable, the conflicting preconditions, and the units that contributed them.

A unit that declares no preconditions contributes nothing to the joint-satisfiability check. The compiler has nothing to check against. This is a structural feature of the mechanism: the substrate does not invent preconditions for the operator. The implications are developed in §5.1 below, where Robodebt’s failure mode is exactly the admission of an algorithm that declares no precondition on the variability its averaging step assumes.

3.1.5 Calibration-handling validation

The pipeline validates the unit’s calibration-handling declaration for well-formedness. A behaviour-characterised unit’s content declares a confidence section: whether the unit produces a calibration value at runtime, the unit’s calibration claim, and the acceptance band against which drift is measured. A policy unit’s content may declare a calibration-gate section: a threshold below which the policy refuses regardless of substantive condition. A higher-order unit’s content declares a propagation function over its sub-units’ calibration values: minimum, product, or another scalar function the architecture admits.

The propagation function is not applied at compilation; it is recorded in the unit’s content, and the runtime applies it. Compilation validates that the declaration is well-formed: the named output field exists in the unit’s output specification, the calibration claim is expressed in terms compatible with the acceptance band, the propagation function is one the architecture admits.

3.1.6 Compiled-form emission and witnessing

The pipeline emits the compiled form and witnesses it under the custodian supplied for the compilation. For a single-operator unit, the witness is the operator’s own custodial arrangement. For a unit compiled under a cooperative substrate, the witness is the cooperative substrate’s quorum custodian (§3.8). Below the quorum threshold, the witness fails, and the admission is refused.

The compiled form is itself a content-addressed state unit. The architecture maintains three archives: a code archive holding functional and state units (including the executable artefacts functional units reference, and the compiled forms themselves), a credentials archive holding credential unit content, and a ledger holding the record of every committed act. There is no separate state-units archive and no separate compiled-forms archive: a compiled form is placed in the code archive and registered on the operator’s runtime. Subsequent invocations of the unit dispatch against the compiled form’s content identity.

3.1.7 What compile-at-commit does not do

It does not refuse units whose substantive content is wrong. A specification-bounded unit whose specification is permissive compiles cleanly; a behaviour-characterised unit whose calibration claim is loose compiles cleanly; a unit that declares no preconditions compiles cleanly. The substrate’s mechanism is structural visibility, not substantive correctness: the operator’s authored content determines what is admissible; the compiler enforces that determination; the operator’s choices remain the operator’s choices and are recorded as such.

It does not refuse units that compose other units the operator distrusts. Distrust is a policy concern; the operator’s policies bind through the credential graph and govern what compositions are admissible. A unit whose composition violates a bound policy is refused at runtime, not at compile time, unless the policy’s structure admits compile-time evaluation (a precondition rather than a context-dependent rule).

It does not guarantee that compilation completes. Compilation is bounded in resources; pathological compositions may exceed the implementation’s operational limits and produce refusal at admission. Compilation integrity may fail under cooperative witnessing when participating operators produce divergent compiled forms for the same content; the architecture admits this failure mode rather than claiming compilation is guaranteed, and the operator’s authored procedure for investigating witnessing disagreement governs the response.

It does not eliminate the operational cost of system evolution. A substrate whose units evolve frequently incurs compilation cost at every commit; a substrate whose credential graph changes propagates recompilation cascades through the dependency graph; a substrate composing rapidly-evolving behaviour-characterised units accumulates drift state and may incur frequent invalidation and recompilation. The architecture’s response is that the operator’s authored content specifies cascade-depth bounds and recompilation budgets: cascades exceeding the budget trigger explicit institutional approval rather than automatic propagation; cascades within the budget produce extended invalidation latency rather than incorrect operation. The architecture trades enforcement-time correctness for adoption-time refusal under dynamism stress: acts whose underlying compiled forms are pending recompilation refuse rather than executing under uncompiled commitments. Whether this trade is acceptable for a given operator’s domain is the operator’s question, not the architecture’s.

3.2 Roll-up under strictest-binding-wins

The policy units in scope at a unit’s compilation are composed into a single rolled-up policy expression. Where multiple policies address the same governance dimension, the strictest binding becomes the binding for that dimension. The architecture’s commitment is that policies do not silently relax through composition: a unit composing under two policies on the same dimension binds under the stricter of the two, and a unit whose composition would weaken any policy in scope refuses.

Strictness is structural. Where two policies’ constraints are comparable (one is a strict subset of the other along the dimension), the subset is strictest. Where two policies’ constraints are incomparable (neither is strictly stronger than the other), the composition is non-reconcilable: §3.4 applies.

The rolled-up policy expression is part of the compiled form. Runtime evaluation (§3.5) evaluates against the rolled-up policy, not against the constituent policies individually. The rolled-up form is deterministic in the inputs (the set of policies in scope, the strictness relations between them, the operator’s authored content for policy ordering where strictness alone does not resolve the rollup); two conforming implementations compiling the same unit under the same authored content produce rolled-up policies of identical content identity.

What roll-up does not do

It does not synthesise a reconciliation where none exists. Non-reconcilable composition refuses; the substrate is not authorised to decide which of two incomparable policies should govern.

It does not invert the strictness relation under operator preference. An operator may declare an ordering between two policies the strictness rule alone does not resolve; an operator may not declare that a weaker policy supersedes a stricter one in compositions where both bind.

3.3 Wilful inclusion

Wilful inclusion is the rule that every unit a composition transitively depends on must be enumerated in the composition's top-level reference set. The mechanism is operationalised in stage one of the compilation pipeline (§3.1.1); the architectural commitment is that composition by hidden dependency is structurally refused.

The rule has two faces. First, references: every unit reachable through any chain of references must appear in the source unit's own top-level reference set. A reference reachable only transitively, through an intermediate unit, fails the check. Second, governance dimensions: the rolled-up policy stack must address every governance dimension the operator's authored content declares applicable to units of this type at this scale. Silence on any required dimension produces compilation refusal.

The rule's purpose is structural visibility. An operator inspecting a unit's content can see the full set of units the composition depends on without traversing the reference graph; an operator inspecting the unit's policy coverage can see whether every required dimension is addressed without inferring from omission. The rule does not prevent the operator from authoring units with extensive dependencies; it requires those dependencies to be wilfully declared.

What wilful inclusion does not do

It does not police the substantive content of the references. A unit whose declared references include units whose substantive content is wrong, malicious, or unwise compiles cleanly under wilful inclusion provided the references are declared. The substrate's mechanism is visibility, not substantive judgement.

It does not detect omitted dependencies the substrate cannot see. If a unit's executable artefact dynamically loads or invokes a unit whose reference is not declared in the unit's content, the wilful-inclusion check passes (the declared reference set is internally consistent) but the runtime invocation refuses (§3.5) because the sub-invocation's compiled form is not present in the parent's wilfully-included set.

3.4 Refusal under non-reconcilable composition

The architecture's response to compositions that cannot proceed is structural refusal, not silent reconciliation. Refusal is a first-class output: the refusal is recorded in the ledger, with the specific cause, the units that contributed to the conflict, and the policy or precondition that was violated. The act is paused; the operator's authored content determines the response.

Non-reconcilable composition arises in three places.

At compilation, between policies in scope. Two policies addressing the same governance dimension but incomparable in strictness produce non-reconcilable composition: neither is strictly stronger than the other, and the architecture does not invent a synthesis. The refusal names the non-reconcilable policy pair. Resolution is political: amending one or both policies, adding a reconciliation policy that orders them, or accepting that compositions in scope of both authorities are not admissible.

At compilation, between preconditions in referenced units. The joint-satisfiability check of §3.1.4 produces refusal where the intersection of preconditions on any input variable is empty. The refusal names the variable, the conflicting preconditions, and the units that contributed them. This is the substrate's compile-time response to data whose specification violates an algorithm's stated precondition. (The contrasting case, an algorithm that declares no precondition at all, compiles cleanly; the substrate's response to that case is runtime policy refusal under a separate bound policy, exhibited in §5.1.)

At runtime, between the act's invocation context and the compiled form's commitments. A unit whose compiled form requires operating conditions that the runtime invocation cannot satisfy refuses execution. A behaviour-characterised unit whose acceptance bands have been exceeded by recent drift detection (§3.6) refuses execution. A unit whose compiled form's wilful-inclusion rule required a dimension that the runtime cannot evaluate refuses execution. In each case, the refusal is recorded with the specific cause.

Refusal is structurally preferred to silent reconciliation because reconciliation requires authority the architecture does not have. The substrate is not authorised to decide which of two non-reconcilable policies should govern; that decision is the work of the policies' authoring authorities.

What refusal does not do

It does not propose a resolution. Refusal names the conflict and stops; the resolution is the work of the operator or of the policies' authors.

It does not retry. A refused composition or refused act is recorded as refused; the operator may author a new unit or invoke under different conditions, but the refusal itself stands on the ledger as a first-class act.

It does not eliminate refusal's operational and human cost. A refusal-heavy substrate produces operational friction the operator absorbs; refusals delay services, block transactions, and shift handling burden onto downstream processes. In public-benefit, healthcare, and other systems where the downstream of a refusal is a person waiting, refusal-as-default has real cost to the people the system serves. The architecture's commitment is not that refusal is costless; it is that refusal is structurally visible and attributable. The operator's authored content must include the refusal-handling policy the substrate does not invent: what happens when a composition refuses at compile time, what happens when an act refuses at runtime, how the operator's institutional process routes refusals to manual review, escalation, or remedy. An operator that authors no refusal-handling policy produces a substrate whose refusals accumulate without recourse; the architecture makes the accumulation visible but does not address it.

It does not prevent bypass channels from emerging around it. Systems that refuse aggressively and provide no operator-authored handling route operators toward shadow workflows that operate outside the substrate's witnessing arrangement. The architecture's response is that work performed outside the substrate is visible by its absence from the ledger; the architecture cannot prevent shadow work, but it can make the gap between substrate-witnessed work and total institutional work measurable. Whether the gap is acted on is the operator's institutional question, not the architecture's.

3.5 Runtime evaluation against the compiled form

Every act dispatches against the compiled form of its target unit. The runtime pipeline has five stages: compiled-form retrieval, delegation check, integrity check, rolled-up policy evaluation, ledger commitment.

Compiled-form retrieval looks up the target unit's compiled form by content identity from the code archive. If the compiled form has been invalidated through the uniform invalidation surface (§3.6), retrieval fails and the act refuses.

Delegation check verifies that the credential presented at invocation has authority over the target unit. The credential's authority chain is walked upward; if no credential in its ancestry intersects the target unit's compiled authority chain, the delegation check fails and the act refuses.

Integrity check verifies that the compiled form's content identity matches the cryptographic hash of its canonicalised content; that the witness signature is valid under the custodian named in the compiled form; that the compiled form has not been superseded under the uniform invalidation surface. Each check, failing, refuses the act.

Rolled-up policy evaluation evaluates the compiled form's rolled-up policy against the act's invocation context. The verdict is permit or refuse; the verdict is the verdict the compiled rolled-up policy produces. Conforming implementations do not synthesise verdicts, do not evaluate against alternative policy structures, do not approximate the rolled-up policy's evaluation, and do not admit acts whose compiled form has been invalidated.

Ledger commitment records the act on the operator's ledger: the act's content (the invocation context, the target unit's content identity, the credential presented, the verdict, the rationale), the act's references (the compiled form's content identity, the credential's content identity), and the witness arrangement under the operator's hosting pattern. Refusal acts are recorded equivalently to permit acts; both are first-class.

The runtime is deterministic in its inputs to the extent that the policy language used by the rolled-up policy is itself deterministic in evaluation. The architecture does not specify the policy language; conforming implementations choose a policy language and the architecture's conformance commitment is that, given a deterministic policy language, two implementations evaluating the same compiled form against the same invocation context produce identical verdicts. Implementations that choose non-deterministic policy languages, or that admit non-deterministic constructs (timeouts whose outcome depends on wall-clock state, randomised tiebreakers, evaluation against external state the architecture does not capture in the com-

piled form) produce substrates that are non-conforming on this commitment. The prototype's policy language is deterministic by construction; production conforming implementations that build on existing policy engines (Open Policy Agent, AWS Cedar, Zanzibar-derived systems) inherit those engines' determinism properties and the conformance commitment applies to the extent those properties hold.

What runtime evaluation does not do

It does not re-evaluate the structural work. The compiled form is the runtime's input; the structural work was performed once at compile-at-commit. The runtime applies the compiled form to the invocation context and produces a verdict.

It does not modify the compiled form. The compiled form is immutable; changes to a unit produce a new compiled form with a new content identity, and the runtime dispatches against the new identity once it is registered.

It does not silently degrade. An act whose evaluation cannot complete (the compiled form is invalidated, the credentials archive is unreachable, the policy evaluation is non-terminating under bounded resources) refuses. The refusal is recorded; the operator's authored content determines the response.

3.6 The uniform invalidation surface

A unit's compiled form is invalidated by any of five triggers, each producing a ledger record and an administrative act through the same mechanism. The uniform surface gives the architecture defensive coherence: one pathway carries every invalidation, rather than each invalidation type requiring its own operational profile.

The five triggers are revocation, deprecation, supersession (of credentials or units), drift, and integrity failure.

Revocation invalidates a credential. The credential's content identity is recorded as revoked on the ledger through the operator's standard administrative interface; compiled forms whose authority chain includes the revoked credential are invalidated.

Deprecation invalidates a unit or credential without supersession. The unit is recorded as deprecated; compiled forms referencing the unit are invalidated unless the operator's authored content explicitly admits composition against deprecated units. The latter case is the architectural commitment: deprecation does not retroactively refuse historical acts that ran against the unit while it was active; it prevents new compositions from depending on it.

Supersession invalidates by replacement. A new credential or unit is admitted with a content identity that supersedes a prior identity; compiled forms referencing the prior identity are invalidated. Recompile against the superseding identity is the recovery path; the recompilation cascade propagates through the dependency graph.

Drift invalidates a behaviour-characterised unit whose runtime observation falls outside its declared acceptance band. The drift mechanism observes an output field the unit's specification names; the mechanism maintains a rolling window of observations in runtime state; when the

windowed mean leaves the declared interval the unit is marked drifted and subsequent invocations refuse until an authorised operator resets the drift state.

Drift state lives in the runtime, not in archive content. It is observed from a model's emitted output field, maintained in memory, and rebuilt from observation when the runtime resumes. Drift state does not change a unit's content identity; the unit's content remains as committed, and the unit's compiled form remains as emitted. What changes is the runtime's willingness to dispatch against it.

Integrity failure invalidates a compiled form whose witness signature fails verification or whose content identity does not match its content hash. The compiled form is recorded as integrity-failed; the operator's authored content determines the response, which typically includes investigation of the witnessing arrangement and recompilation.

The complementary mechanism, the backtest pattern, operates on the historical ledger evidence directly. A backtest is a regular functional unit whose implementation walks the operator's ledger, pairs the target unit's predictions with realised outcomes under a declared correlation reference, computes a calibration metric over the paired observations, and refuses if the metric falls outside the declared acceptance band. When the backtest refuses, an authorised operator deprecates the target unit through the standard administrative interface; deprecation invalidates compiled forms referencing the unit through the uniform invalidation surface. The backtest closes the verification loop on the calibration claim against historical evidence; drift closes the loop on recent observations. Together, they make the calibration claim load-bearing.

What the uniform invalidation surface does not do

It does not retroactively refuse historical acts. An act that ran while the unit was active is recorded on the ledger as the act it was; invalidation prevents new acts but does not rewrite history.

It does not invalidate units the operator chooses to retain. An operator may, through authored content, admit composition against a deprecated unit; the deprecation is recorded and the compositions are visible, but the architecture does not enforce a universal deprecation rule.

It does not detect substantive failure the calibration claim and the operator's policies do not catch. An algorithm whose calibration is reported correctly but whose substantive output is misused is not invalidated by drift or backtest; the substantive question is for the operator's authored content and the institutional process it operationalises.

3.7 Administrative acts on the ledger

The operator's administrative interface to the substrate's own governance state is a set of operations the architecture admits: revoke a credential, deprecate a credential or unit, supersede a credential or unit, reset a drift state. Each operation is itself a substrate act, authorised by the operator's delegated credentials, governed by the same architecture as any other act.

The administrative operations are precisely the operations that invalidate existing compiled

forms. They are not the operations that admit new units; admitting a new unit is a workflow (place the unit in the code archive, compile it, register the resulting compiled form on the operator's runtime) that produces a content-addressed compiled form, and the ledger records each subsequent invocation against the new identity but does not record the commit itself as an administrative act.

This is the substrate's self-hosting property: the operator's administrative actions are governed by the same architecture the operator is using. The substrate does not exempt its own administrative interface from its own architectural commitments. An administrative act produces a ledger record; the record is content-addressed; the witnessing arrangement under the operator's hosting pattern applies.

Conforming implementations expose the administrative operations through whatever interface they choose; the architectural commitment is that the operations are substrate acts under the architecture's commitments. An implementation that admits administrative operations outside the ledger, or that bypasses witnessing for administrative operations, is non-conforming.

What administrative acts do not do

They do not include commit-of-a-new-unit. Committing a unit is a workflow that produces a compiled form; the ledger records invocations against the compiled form, not the commit itself.

They do not invent authority. An administrative operation requires a credential whose authority covers the operation; the delegation check applies as it does for any act.

They do not relax the architecture's commitments. An operator's administrative interface is not an escape hatch from the architecture; it is a subset of the architecture's act vocabulary.

3.8 Federation of archives under multi-custodian quorum

Operators compose into cooperative substrates by joint authority over shared archives. The cooperative substrate is itself a substrate at composite scale: its authored content is jointly authored by its member operators; its archives are the shared archives the cooperation requires; its quorum custodian composes the member operators' individual custodial arrangements into a joint witness requiring the cooperative substrate's declared threshold of member signatures.

Each archive is itself a substrate. The architecture admits four hosting patterns. Single self-hosted: the operator runs the archive's hosting alone, and the archive's defensive properties are whatever the operator's internal arrangement produces. Single delegated: the operator delegates archive hosting to a chosen third party, and the defensive properties are whatever the third party's infrastructure produces. Cooperative non-delegated: the originating operators jointly host the archive, and the defensive properties are the diversity of the cooperative substrate's member operators along jurisdictional, institutional, and technical dimensions. Federated: the originating operators grant operator status over the archive to a cooperative substrate of trusted third parties chosen for the purpose, and the defensive properties are the diversity of the federation's member operators constituted specifically for the purpose.

Defensive properties scale with the pattern. Single self-hosting produces effectively no archi-

tectural defensive properties beyond what content-addressing and append-only commitment produce within the operator's own infrastructure. Cooperative non-delegated produces defensive properties commensurate with the cooperative substrate's member diversity. Federation produces the substrate's strongest defensive properties; it also requires the most institutional work to establish and maintain. The architecture is honest about the scaling. An operator under single self-hosting cannot expect federation-pattern defensive guarantees from their archive's existence.

Each archive's pattern is independent. An operator may run the credentials archive under one pattern, the code archive under another, and the ledger under a third; the choice is per-archive, per-operator. The choice may evolve over time: starting with single self-hosting at adoption, moving to cooperative non-delegated as cooperative substrates form, moving to federation as the threat model justifies the institutional investment.

The protocol's commitments around archive content are uniform across the patterns. Signature schemes are specified with explicit migration paths; quantum-resistant primitives are required, with specific algorithms chosen by conforming implementations from the standards their jurisdictions recognise; rotation and migration windows are part of conformance. Integrity guarantees are specified at the protocol level: the ledger uses witnessed log structures with multi-custodian quorum commitment where the pattern admits multi-custodian witnessing; the credentials archive maintains historical versioning with cryptographic integrity at every version; the code archive supports content-addressable retrieval with bit-identical guarantees.

Adversarial and failure assumptions

The architecture inherits its witnessed-log model from RFC 9162 Certificate Transparency. The adversarial assumption is that an attacker may compromise some witnessing operators but not enough to defeat the quorum threshold the cooperative substrate's authored content specifies. Under cooperative non-delegated and federated hosting patterns, this is a Byzantine fault tolerance commitment: the substrate tolerates Byzantine behaviour from a strict minority of witnessing operators below the quorum threshold and refuses admission when the quorum threshold cannot be reached. The specific threshold is the cooperative substrate's authored content, not an architectural constant; conformance is that the threshold is enforced structurally rather than negotiated per admission.

Partition tolerance is bounded by the quorum requirement. A witnessing operator partitioned from a quorum sufficient to admit cannot admit; the admission refuses or is delayed pending partition recovery. The architecture trades liveness under partition for safety under partition: the substrate prefers to refuse admission during partition rather than admit under reduced quorum, which would weaken the defensive properties the pattern produces. Conforming implementations may expose operator-configurable timeouts on partition recovery; the timeout's expiry produces a refusal act, not a silent admission.

Liveness guarantees are properties of the chosen custodian implementation, not of the architecture. The architecture commits that admissions either succeed under quorum or refuse with cause; it does not commit that admissions succeed within bounded time under arbitrary net-

work conditions. Production conforming implementations would specify operational liveness commitments appropriate to their domain (financial clearing windows, certification cycles, audit deadlines); the architecture does not impose a single liveness budget.

Witness recovery follows the architecture's existing mechanisms. A witnessing operator whose key material is compromised produces a credential revocation through the standard administrative interface (§3.7); historical attestations the compromised key produced are invalidated through the uniform invalidation surface (§3.6), with the operator's authored content determining the response to bulk invalidation. The substrate admits the recovery pattern; it does not eliminate the institutional work of investigating compromise and re-establishing custodial diversity.

What federation does not do

It does not produce trust where none exists. The cooperative substrate's defensive properties depend on member-operator diversity; a cooperative substrate of operators whose interests align and whose institutional, jurisdictional, and technical characteristics are similar produces weaker defensive properties than a cooperative substrate of operators whose diversity is substantial.

It does not survive quorum collusion. Coordinated compromise of operators participating in an archive substrate that exceeds the substrate's quorum threshold defeats the architecture's defensive properties for affected acts. The defence is custodial diversity sufficient that compromising the quorum requires capabilities no single state actor possesses; the architecture is honest that this is a defence-in-depth, not a guarantee.

It does not produce institutional willingness to form cooperative substrates. The architecture specifies the mechanism for cooperative substrates and the defensive properties each hosting pattern produces; it does not produce the political and institutional commitments that bring cooperative substrates into existence. An architectural specification of joint witnessing does not, by itself, cause regulators and the regulated to jointly witness. The monograph's adoption treatment develops this; the architecture provides the mechanism, not the will.

It does not maintain itself. Diversity is maintained by ongoing institutional work by the participating operators: monitoring for diversity erosion, responding to threats against the diversity arrangement, evolving the authored content as cryptographic stacks deprecate and jurisdictional alignments shift. The cost is real and continuing; the benefit is that the archive's defensive properties are not merely asserted at deployment but maintained operationally.

4. The reference implementation

The reference implementation accompanies this paper at <https://github.com/swheeler-research/substrate-reference>. It comprises roughly three and a half thousand lines of core substrate code in `src/substrate/`, with a further roughly four and a half thousand lines of tests and roughly nine thousand lines of worked examples; it exercises 225 tests and provides twelve end-to-end demonstrations. The implementation is a reference, not the conforming implementation; production conforming

implementations will exercise the same architecture with different engineering choices about durability, performance, key management, and operational tooling.

4.1 Module correspondence to the specification

The implementation's modules realise the specification of §3 as follows.

`src/substrate/primitives.py` defines the three primitive role types and their internal typings (contract patterns for functional units, mutability disciplines for state units, transfer disciplines for credential units). §2 of this paper specifies what these roles are; the module specifies the type signatures conforming implementations may exchange.

`src/substrate/contracts.py` provides the structural contract language for unit content: input and output specifications, preconditions, calibration claims, drift criteria, propagation functions. §2.1 specifies what these declarations carry; the module specifies their structural form.

`src/substrate/compile.py` is the compilation pipeline. The six stages of §3.1 run in the order specified: wilful inclusion check, authority chain resolution, policy collection, joint-satisfiability check on structured preconditions, calibration-handling validation, compiled-form emission and witnessing.

`src/substrate/operator.py` is the `Operator` class. It exposes the administrative operations of §3.7 (`revoke_credential`, `deprecate_credential`, `supersede_credential`, `deprecate_unit`, `reset_drift`); these are the only operations the class exposes as administrative acts. Committing a new unit is a workflow (place, compile, register) the class supports as method calls but does not record on the ledger as an administrative act.

`src/substrate/runtime.py` is the `Runtime` class. It performs the five-stage invocation pipeline of §3.5 (compiled-form retrieval, delegation check, integrity check, rolled-up policy evaluation, ledger commitment) at every act.

`src/substrate/ledger.py` is the hash-chained ledger. It records every act, invocation and administrative, with content addressing and hash-chained integrity.

`src/substrate/federation.py` is the cooperative substrate and quorum witnessing implementation. It composes member operators' custodial arrangements into the joint witness of §3.8 and applies the quorum threshold the cooperative substrate's authored content specifies.

`src/substrate/drift.py` is the drift detection mechanism of §3.6. It observes the named output field on each invocation, maintains the rolling window in runtime memory, and marks the unit drifted when the windowed mean leaves the declared interval.

`src/substrate/confidence.py` is calibration as first-class architectural property. It records the calibration claim and acceptance band in the unit's content (validated at compilation under §3.1.5), produces the calibration value at runtime, and applies the propagation function the unit's content declares.

`src/substrate/backtest.py` is the backtest pattern of §3.6. It provides reference implementations of four canonical calibration metrics that backtest units may declare in their content: coverage rates over prediction intervals, calibration-error metrics over probability outputs,

exceedance rates over value-at-risk bounds, classification-accuracy metrics over labelled outcomes.

`src/substrate/implementations.py` holds the content-addressable functional unit implementations the demonstrations exercise.

The demonstrations live under `examples/`, each in its own directory with `run.py` (the demonstration itself) and `README.md` (case-study text mapping the demonstration to substrate mechanisms). The twelve demonstrations are: Universal Credit (single operator), Universal Credit (cross-operator), Horizon, Robodebt, Lavender, SolarWinds, CrowdStrike, Boeing 737 MAX, Five Eyes, LIBOR, London Whale, and Constitutional anchoring. The tests under `tests/test_*.py` exercise each module independently; the demonstrations exercise the modules in composition.

4.2 What the prototype does not yet substantiate

The architecture specifies six commitments the reference prototype does not yet substantiate at the engineering depth a production conforming implementation would require. These are gaps in the prototype, not gaps in the architecture; the architectural commitments are stable.

Extraction-maturity level as structured unit content. The architecture specifies that a unit's content declares its extraction maturity level (Level 0: wrapped legacy; Level 1: boundary-decomposed; Level 2: functionally extracted; Level 3: reimplemented with formal proof) and that an operator's authored content may specify a minimum maturity level for units it admits. The prototype admits units without enforcing maturity-level declarations.

Observability provenance as structured unit content. The architecture specifies that a unit's content declares what it can observe about its own operation, what it cannot, and what its declared blind spots are; policies that require observability the unit cannot provide refuse composition at compile time. The prototype admits units without enforcing observability-provenance declarations.

Visibility class for policy content. The architecture specifies that policy content carries a visibility class (under what conditions the policy's content is disclosed to invokers, auditors, regulators) and that the runtime enforces the visibility class at policy disclosure. The prototype treats all policy content as uniformly visible.

Twinning of constitutional source credentials with guardian-quorum revocation. The architecture specifies that a constitutional source credential's revocation requires a guardian quorum composed of natural persons whose identities are bound through institutional and legal apparatus to the constitutional process the source represents. The prototype admits constitutional source credentials but does not implement the guardian-quorum revocation pattern.

Persistent drift state surviving prototype restart. The drift mechanism maintains a rolling window in runtime memory; on prototype restart, the window is rebuilt from observation. A production conforming implementation would persist the drift state across restart with the same integrity properties as ledger content. The prototype rebuilds from observation.

Calibration-handling contract as first-class field on the compiled form. The calibration claim and acceptance band live in the unit's content and are validated at compilation; the com-

piled form references this content. A production conforming implementation would emit the calibration-handling contract as a first-class field on the compiled form, machine-checkable without traversing back to the unit's content. The prototype validates at compilation but does not promote the contract to a compiled-form field.

These gaps are named explicitly because the prototype is reference, not production. The demonstrations that exercise the prototype's existing engineering depth substantiate the architectural mechanisms specified in §3; the gaps are work for production conforming implementations to do.

4.3 Conformance and interoperability

Conforming implementation is what makes the architecture's defensive properties hold operationally. An implementation can satisfy the architecture's mechanical requirements while still producing substrates that interoperate with other conforming substrates across operator boundaries. An implementation that fails the conformance requirements produces a substrate whose acts cannot be jointly witnessed in a cooperative substrate with conforming counterparties; the non-conforming substrate is isolated by the architecture's own admission discipline.

The conformance commitments are precisely those specified in §3. At unit commit time, the six stages of §3.1 are required. At every act, the five-stage runtime evaluation of §3.5 is required. The uniform invalidation surface of §3.6 is honoured. The runtime's verdict is the verdict the compiled rolled-up policy produces; conforming implementations do not synthesise verdicts, do not evaluate against alternative policy structures, do not approximate the rolled-up policy's evaluation, and do not admit acts whose compiled form has been invalidated.

Implementations have substantial latitude on how they satisfy these requirements. Execution technologies, optimisation strategies, federation topologies, performance and cost profiles, caching disciplines, batching arrangements where unit policies admit them, and parallelism choices are implementation decisions. The architecture cares about results, not means. What the architecture does not admit is implementations whose results differ from the canonical evaluation under the determinism precondition of §3.5: implementations whose policy languages are themselves deterministic, and that do not admit non-deterministic constructs, satisfy the conformance commitment that identical compiled forms produce identical verdicts on identical invocation contexts.

The runtime layer is structurally adjacent to existing production policy engines (Open Policy Agent, AWS Cedar, Zanzibar-derived systems); the architecture's contribution is the composition discipline above the runtime layer, not the runtime itself. Conforming implementations may build on these substrates where their semantics admit; the architecture does not prescribe the policy-engine implementation, and the determinism conformance commitment applies to the extent the chosen engine's evaluation is deterministic.

4.4 What the prototype demonstrates about cost

The prototype substantiates the architectural mechanisms of §3 against twelve worked cases. It does not substantiate the architecture's tractability at production scale. The distinction matters for the cost questions a serious conforming implementation must answer.

What the prototype demonstrates. The six compilation stages of §3.1 complete in bounded time on each of the twelve demonstrations' unit graphs, which range from approximately ten units (Universal Credit single operator) to approximately thirty units (London Whale) per demonstration. Runtime evaluation per act completes in bounded time and produces deterministic verdicts under the prototype's policy language. The drift mechanism's rolling-window observation per act has constant cost in the window size. The backtest pattern's ledger walk has cost linear in the ledger length over the backtest's correlation window. The invalidation surface's cascade depth is bounded in the demonstrations by the depth of the dependency graph, which is small.

What the prototype does not demonstrate. Asymptotic complexity of compilation under large policy graphs with many cross-referenced credential chains. Runtime evaluation cost under policy stacks an order of magnitude larger than the demonstrations exercise. Invalidation propagation cost under cascade depths characteristic of institutional substrates with hundreds or thousands of units. Witness coordination cost under cooperative substrate quorums larger than the bilateral arrangements the demonstrations model. Performance under adversarial inputs designed to maximise compilation or evaluation cost. Memory and storage growth of the archive substrates over operational time scales of years.

What this implies for conformance. Production conforming implementations require benchmark work the prototype does not perform: compilation cost as a function of policy graph size; runtime evaluation cost as a function of rolled-up policy depth and rule count; invalidation cascade cost as a function of dependency graph depth and breadth; witness coordination latency as a function of quorum size and partition characteristics; storage growth as a function of operational throughput. The benchmarks are not the architecture's commitments; they are the conforming implementation's commitments under its chosen engineering profile. The architecture commits to the mechanism; the implementation commits to the cost.

The architectural commitment that compilation is amortised across invocations holds by construction: the compiled form is content-addressed and immutable, and runtime evaluation against it does not re-execute the structural work. The commitment that runtime evaluation is cheap relative to compilation holds to the extent that the chosen policy language is evaluable in cost proportional to the rolled-up policy's size, which is a property of the policy engine the implementation chooses. The architecture does not impose a policy language and therefore does not impose a runtime-evaluation cost bound; the choice of policy engine is part of the conforming implementation's specification, and the runtime cost claim is bounded by that choice.

The honest framing for a production conforming implementer is that the prototype substantiates the mechanisms; the engineering of cost-bounded production substrates is the work the implementer does, against the policy engine, storage backend, witnessing infrastructure, and operational profile the implementer's domain requires.

5. The architecture under exercise

This section exhibits the architecture running. Three cases are developed at sufficient depth to show the mechanisms of §3 composing against substantively different problems without bending: Robodebt (the runtime-policy distinction), Boeing 737 MAX (certification as a substrate pattern), and London Whale (the densest composition of mechanisms in the case set). The cases run in the reference implementation; the substantive content beyond what is needed to exercise the architecture lives in the monograph and in the demonstrations' `README.md` files.

5.1 Robodebt

The Australian Robodebt scheme between 2015 and 2019 issued debt notices to welfare recipients on the basis of an algorithm that averaged annual income across fortnightly periods, presuming roughly steady income as a precondition for valid comparison. Gig-economy and casual workers' income violated the precondition; the algorithm produced phantom debts; the scheme reversed the burden of proof onto the recipient; over 470,000 wrongful debts were issued before the scheme was halted; the resulting Royal Commission found the scheme had been operationally unlawful.

The substrate deployment models three operators federated under two cooperative substrates. Services Australia and the Australian Taxation Office compose under a data cooperative substrate for income-data exchange; Services Australia and the Commonwealth Ombudsman compose under an audit cooperative substrate for oversight access. Two versions of the debt-calculation algorithm exist as distinct content-addressed functional units. `compute_debt_v1` is the Robodebt-shaped algorithm: its content declares no preconditions on income variability, and no variability policy binds it. `compute_debt_v2` is a revised algorithm whose references include an `income_variability_policy` that refuses inputs whose coefficient of variation exceeds a declared bound.

The substrate's behaviour is the architectural point. `compute_debt_v1` is a well-formed unit. The compilation pipeline runs through its six stages: the wilful-inclusion check passes (the unit's references are internally consistent); authority chain resolution terminates at the Australian constitutional source; policy collection assembles whatever policies are otherwise in scope; the joint-satisfiability check encounters no preconditions to evaluate (the unit declares none, so the check has nothing to refuse); calibration-handling validation passes (the unit is specification-bounded, not behaviour-characterised); compiled-form emission and witnessing succeeds under Services Australia's custodial arrangement. The unit compiles. Invoked against a steady-income claimant, `compute_debt_v1` returns a defensible figure. Invoked against a gig-economy claimant whose fortnightly income is highly variable, it returns a phantom debt, exactly as the historical scheme did.

`compute_debt_v2` is a different unit with a different content identity. Its compilation runs through the same six stages; the difference is that its references include the variability policy, which is collected into the rolled-up policy stack at stage three. The unit compiles. Invoked against the same gig-economy claimant, the runtime evaluation pipeline retrieves the compiled form, walks the credential's authority chain, verifies integrity, evaluates the rolled-up policy against the invocation context. The variability policy refuses: the act is a first-class refusal with a

structurally visible rationale naming the policy and the variability evidence in the act's content. The refusal is committed to Services Australia's ledger; the act is paused; the operator's authored content determines the response, which under appropriate authoring is the case routing to manual review under caseworker authority.

The architectural point of the case is twofold. First, the substrate does not invent preconditions for the operator. An algorithm that declares no precondition presents the compiler with nothing to refuse; the unit compiles; the failure mode of the historical scheme is reproducible under the substrate when the precondition is absent. The substrate's mechanism is not substantive correction but structural visibility. Second, what the substrate makes structural is the contrast and its visibility. Every debt figure on either version's ledger entries records the content identity of the version that produced it; an audit can establish, per claimant, whether the figure came from an algorithm carrying the precondition or one carrying none. The Ombudsman's cross-operator audit access, through the audit cooperative substrate, lets oversight reconstruct this from the archive substrates' content without depending on Services Australia's cooperation. The choice between the two algorithms, and the consequences of that choice for each claimant, are structurally legible rather than buried in operational opacity.

The complementary compile-time path, in which an algorithm that declares preconditions cannot be admitted against a data source whose output specification violates them, is exercised in the Universal Credit demonstration (`examples/universal_credit/`). The substrate's compile-time refusal under joint-satisfiability failure is the response to that case; Robodebt is the response to the case where compile-time refusal cannot apply because the unit declares no preconditions, and the response is runtime policy refusal under a bound policy.

The mechanisms of §3 exercised: §3.1 (compile-at-commit, all six stages on both units), §3.2 (roll-up of the variability policy into the v2 compiled form's rolled-up policy), §3.3 (wilful inclusion across both units' reference sets), §3.4 (refusal under non-reconcilable composition at runtime, when the variability policy refuses), §3.5 (runtime evaluation of both compiled forms), §3.7 (no administrative acts in this case; both units are committed and run), §3.8 (cooperative-substrate hosting of the shared archives between Services Australia, ATO, and the Ombudsman).

5.2 Boeing 737 MAX

The Boeing 737 MAX's Maneuvering Characteristics Augmentation System (MCAS) failed in October 2018 (Lion Air 610) and again in March 2019 (Ethiopian Airlines 302), killing 346 people across the two crashes. The failures are documented by the National Transportation Safety Board, the Joint Authorities Technical Review, and the United States Congressional Transportation Committee investigation. The structural elements of the failure relevant to the substrate are: MCAS relied on a single angle-of-attack sensor where the aircraft had two; pilots were not informed of MCAS's existence in the initial flight-crew operating manual; certification used an amended type certificate process under which Boeing's Designated Engineering Representatives performed much of the safety analysis Boeing was being certified on; and inter-airline evidence sharing after Lion Air 610 was structurally limited, so Ethiopian's deployment proceeded without the canary's evidence.

The substrate deployment models four operators federated under a cooperative substrate for certification: Boeing (manufacturer), the FAA (regulator), Lion Air (canary airline), Ethiopian Airlines (production airline). Two versions of MCAS exist as distinct functional units. `mcas_v1_single_sensor` declares one angle-of-attack sensor in its input specification and reads only the left sensor in its implementation. `mcas_v2_dual_sensor` declares both sensors, reads both, declares a calibration claim on sensor agreement (1.0 at zero disagreement; dropping linearly to 0 at 5 degrees of disagreement), and references a `captain_authority` credential as part of its authority chain. The FAA authors two policy units: `multi_sensor_required_policy` refuses if the candidate unit's specification declares fewer than two sensors; `pilot_override_required_policy` refuses if the candidate's authority chain does not include the pilot credential. The FAA also authors a functional unit, `certify_mcas`, whose implementation fetches a candidate unit's content, inspects its declared structure, and invokes the two policy units as sub-units.

The architectural point of the case is that certification is a substrate pattern distinct from both compile-time refusal and runtime invocation policy. `mcas_v1` is well-formed; the compilation pipeline runs through its six stages and emits a compiled form. The unit compiles. What `mcas_v1` fails is certification, which is a separate witnessed act: the FAA invokes `certify_mcas` against `mcas_v1` as the target unit; `certify_mcas` fetches the candidate's content; the `multi_sensor_required_p` sub-invocation refuses because the candidate's specification declares one sensor; the certification act is committed to the FAA's ledger as a refusal, with the rationale naming the policy and the deficient declaration.

Certification is neither compile-time nor runtime invocation policy. It is a functional unit that examines another unit's structure. It is parallel to the backtest pattern (§3.6): a backtest is a functional unit whose implementation walks the ledger and pairs predictions with outcomes; a certification is a functional unit whose implementation fetches a candidate unit and inspects its declared structure. Neither is a new primitive. Both are regular functional units exercising the substrate's existing composition machinery against their respective evidence sources. The certification unit `certify_mcas` is itself compiled under the cooperative substrate's quorum custodian (§3.8), so Boeing cannot produce it alone: the manufacturer plus regulator plus airlines jointly witness the certification machinery's admission.

`mcas_v2` passes certification. `certify_mcas` invokes the two policy units against the candidate; the multi-sensor policy permits (two sensors declared); the pilot-override policy permits (captain credential present in the authority chain); the certification act is committed to the FAA's ledger as a permit, and `mcas_v2` is registered on the airline runtimes for deployment. Lion Air operates first as canary; the prototype runs five flights with normal angle-of-attack readings; `mcas_v2`'s calibration claim is observed; the airline's `fleet_observation_report` (a functional unit aggregating ledger acts) is exposed under the cooperative substrate. Ethiopian queries the report cross-operator under the cooperative substrate before deploying.

On a flight with faulty sensor readings (left 75°, right 6°), `mcas_v2`'s calibration value drops below the actuation threshold under its declared composition function; the runtime's policy evaluation refuses the nose-down command and surfaces the refusal as the architectural refer-to-human signal. The pilot retains control. This is the architectural moment Lion Air 610 and Ethiopian 302 did not have.

What the substrate does and does not prevent in this case. It does not prevent a manufacturer publishing a faulty implementation; the substrate does not author the manufacturer's units. What it does is prevent the implementation reaching certification (the certification unit refuses), reaching the fleet without joint witnessing (the certification unit is compiled under cooperative-substrate quorum), and reaching multiple airlines without the canary's evidence propagating (the cross-fleet observation report is structural).

The mechanisms exercised: §3.1 (compile-at-commit; both `mcas` units compile cleanly), §3.3 (wilful inclusion; `mcas_v2` declares the captain credential in its authority chain references), §3.5 (runtime evaluation; the refusal on sensor disagreement is policy evaluation against the rolled-up policy), §3.6 (calibration as first-class architectural property; the calibration value composes through the unit's declared propagation function), §3.7 (the certification act is committed to the FAA's ledger through the standard administrative interface for ledger acts), §3.8 (cooperative-substrate quorum custody of the certification machinery).

The institutional anchoring of natural persons to specific FAA certifiers, Boeing engineers, and airline pilots is outside what the prototype substantiates; the architectural mechanism for constitutional source credentials exists, and the institutional and legal apparatus that binds key-pairs to natural persons is the work the monograph names as outside the architecture's scope.

5.3 London Whale

The London Whale demonstration models the 2012 JPMorgan Chase Chief Investment Office synthetic-credit-derivatives loss of approximately USD 6.2 billion. The structural elements relevant to the substrate are: a value-at-risk model whose calibration had drifted from the portfolio's actual behaviour; a recalibrated model substituted for the original, producing systematically lower risk figures on the same portfolio inputs; trading positions that exceeded desk limits without an architecturally visible escalation; backtesting that did not invalidate the recalibrated model when its predictions diverged from realised outcomes; and a regulator (the OCC) whose reconstruction of what had occurred depended on cooperation from the institution whose conduct was being investigated.

The substrate deployment models two operators, JPMorgan and the OCC, federated under a cooperative substrate for audit. Each operator runs its own substrate composed of the three primitive archives plus an append-only ledger. The cooperative substrate is established through bilateral mutual recognition: JPMorgan's authored content admits OCC as a counterparty for audit; OCC's authored content admits JPMorgan as a counterparty for supervisory access.

The case is the densest exercise of mechanisms in the case set. The trace runs in seven rounds, each developed below.

5.3.1 Routine position under the original VaR

The original VaR model (`var_model_v1`, a behaviour-characterised functional unit declaring a drift criterion of windowed mean realised-over-predicted volatility in $[0.8, 1.5]$ over four observations) is committed to JPMorgan's substrate. The trader requests a position within the desk's authorised limit; `authorise_position` (a higher-order functional unit composing the VaR model output with the `position_limit_policy`) is invoked; the runtime evaluation pipeline

retrieves its compiled form, verifies the trader credential, evaluates the rolled-up policy; the position is permitted; the act is recorded on JPMorgan’s ledger. The drift module observes the VaR model’s output field for the realised-over-predicted ratio; the rolling window has one observation; the window’s mean is within the declared interval; the unit remains undrifted.

5.3.2 Recalibration

JPMorgan commits the recalibrated VaR model (`var_model_v2`) as a new content-addressed unit. The compilation pipeline runs through its six stages; the new model compiles. `var_model_v2` has a distinct content identity from `var_model_v1` because its content differs; the substitution is not silent. The same `authorise_position` higher-order unit cannot reference `var_model_v2` without recompilation, because `var_model_v1`’s content identity is what `authorise_position`’s compiled form references. A new `authorise_position` is committed that references `var_model_v2`; this new `authorise_position` has a distinct content identity. Both versions are visible on the ledger; an audit can establish which version produced which act.

5.3.3 Drift

Positions continue under the new `authorise_position` referencing `var_model_v2`. The drift module observes `var_model_v2`’s output field on each invocation; the rolling window fills; on the fourth observation of windowed mean outside $[0.8, 1.5]$, the drift module marks `var_model_v2` drifted in runtime state. The act that produced the fourth violation is recorded normally; subsequent invocations of `var_model_v2` (and therefore of the new `authorise_position` that references it) refuse with rationale citing the drift state.

5.3.4 Over-limit position refused

A trader requests an over-limit position. The `authorise_position` unit’s rolled-up policy includes the `position_limit_policy`; the rolled-up policy evaluation refuses; the act is a first-class refusal on the JPMorgan ledger with rationale naming the policy and the over-limit input.

5.3.5 Escalation

A senior risk officer presents an escalation credential alongside the trader credential. The `position_limit_policy`’s content admits the over-limit position under the senior-risk-officer credential; the rolled-up policy evaluation permits the act with the escalation recorded on the act itself; the escalation is attributable to the named senior officer through the credential’s content identity, not an implicit override.

5.3.6 Backtest after drift reset

An authorised operator at JPMorgan invokes `reset_drift` (an administrative operation under §3.7) on `var_model_v2`; the drift state is reset; the unit is again available for invocation. Independently, a `backtest_var_calibration` functional unit is committed and run. Its implementation walks JPMorgan’s ledger, pairs the predictions `var_model_v2` produced (extracted from invocation records) with realised outcomes (the realised P&L recorded on subsequent position-resolution acts), computes the exceedance-rate metric against the declared value-at-risk bound, and refuses because the computed exceedance exceeds 5%. The backtest refusal

is a first-class act on the ledger. An authorised operator deprecates `var_model_v2` through the standard administrative interface (§3.7); the deprecation invalidates compiled forms referencing the model through the uniform invalidation surface (§3.6); subsequent attempts to invoke `authorise_position` (which references `var_model_v2`) refuse pending recompilation.

5.3.7 OCC audit

OCC invokes `investigate_bank` (a functional unit under OCC’s authority chain) against JPMorgan’s substrate. The cooperative substrate’s audit credentials authorise OCC to retrieve the lineage of compiled forms, ledger acts, drift events, backtest refusals, position-limit escalations, and deprecation events from JPMorgan’s archives. The audit produces a forensic report (itself a state unit committed to OCC’s archive substrate, witnessed under the cooperative substrate’s pattern). The report’s substantive content (was the recalibration appropriate? was the escalation use proportionate? was the backtest’s refusal acted on with appropriate speed?) is OCC’s substantive judgement work. The architectural contribution is that OCC has the substantive material on which the judgement can rest, reconstructable from cryptographically attested artefacts independently of JPMorgan’s cooperation.

The mechanisms exercised, in one case: §3.1 (compile-at-commit on the VaR models, `authorise_position`, the backtest); §3.2 (roll-up of `position_limit_policy` into `authorise_position`’s compiled form); §3.3 (wilful inclusion across the references of all units); §3.4 (refusal under the over-limit position without escalation); §3.5 (runtime evaluation on every position act, every drift observation, every backtest invocation); §3.6 (drift detection on `var_model_v2`; backtest refusal; the uniform invalidation surface routing drift to compiled-form invalidation, backtest refusal to deprecation, deprecation to recompilation cascade); §3.7 (the administrative operations: `reset_drift`, `deprecate_unit`); §3.8 (cooperative-substrate hosting of the audit relationship between JPMorgan and OCC; multi-operator witnessing of cooperative-substrate acts).

The institutional and legal apparatus binding constitutional source credentials to the JPMorgan board and the OCC’s statutory authority is outside what the prototype substantiates. The architectural mechanism for constitutional source credentials exists; the institutional anchoring is the work the monograph names as outside the architecture’s scope.

What the case verifies. Calibrated reliability metadata is architectural: every behaviour-characterised functional unit declares its calibration claim and acceptance band; the substrate carries the declaration as part of the unit’s identity; substituting a more lenient calibration produces a structurally different unit. Drift detection at runtime invalidates the model when observed behaviour falls outside the acceptance band; the backtest pattern walks the ledger to verify the calibration claim against realised outcomes and triggers deprecation through the administrative interface when the claim fails. Position-limit policy with credentialed escalation makes the over-limit authorisation a structural ledger event attributable to a specific named senior officer, not an implicit override of an external risk constraint. Cross-operator audit reconstructs the calibration history, the drift events, the backtest refusals, the position-limit escalations, and the deprecation events from the archive substrates’ content; the regulator’s ability to reconstruct what happened does not depend on the operator’s cooperation.

6. What the architecture does not guarantee

The architecture's defensibility depends on engagement with what it does not do. The limits below are not concessions; they are the architecture's structural commitments about its own scope. A serious architectural specification states its limits where its mechanisms are specified; this section concentrates the limits the §3 subsections each name at the point of mechanism specification, plus the limits that arise from the architecture's relation to existing work.

The architecture does not solve politics. The substrate operationalises what constitutional process, institutional judgement, professional practice, and political deliberation produce; it does not produce them. An operator whose authored content embeds substantively unjust policies produces a substrate that runs those policies attributably; the substrate's contribution is that the choice and its consequences are structurally visible, not that the choice is good.

The architecture does not prevent coercive sovereigns. A sovereign that compromises a cooperative substrate's quorum, captures the constitutional process anchoring its operators, or excludes counterparty operators from cooperative arrangements produces architecturally-recognised acts whose substantive illegitimacy may surface only through their effects. The architecture's defence is custodial diversity sufficient that compromising the quorum requires capabilities no single state actor possesses; the architecture is honest that this is a defence-in-depth, not a guarantee.

The architecture does not solve semantic undecidability. Rice's Theorem establishes that non-trivial semantic properties of programs are undecidable in general. The substrate's response is that the architecture admits behaviour-characterised contracts where specification-bounded contracts are not available, and admits calibration claims with acceptance bands rather than exact correctness assertions; the architecture's strongest claims hold for the structural properties of composition, not for the semantic correctness of the units composed.

The architecture's defensive properties scale with adoption. A small number of conforming implementations under thin cooperative arrangements provide weaker defensive properties than a large number operating under cooperative or federated arrangements with substantial diversity. The substrate's adoption path is part of its defensive arrangement; early adoption produces lower defensive value than late adoption, with defensive value scaling as participation grows.

The architecture's defensive properties scale with the chosen archive hosting pattern (§3.8). A substrate under single self-hosted patterns provides the weakest defensive properties; a substrate under federated patterns with substantial diversity provides the strongest. The architecture does not pretend that single-operator patterns deliver federation-pattern guarantees.

The architecture does not refuse units whose substantive content is wrong. An algorithm that declares no precondition, a calibration claim that is loose, a policy that is permissive, a credential whose scope is broad: each compiles cleanly under wilful inclusion provided the declaration is internally consistent. The substrate's mechanism is structural visibility, not substantive correction.

The architecture does not detect compromise the counterparts fail to detect. A compromised

operator that produces architecturally-legitimate acts whose substantive illegitimacy is invisible to the cooperative substrate's counterparts is not surfaced by the substrate. The architecture's response is the cooperative substrate's quarantine, recovery, and excision mechanisms once compromise is detected; the architecture cannot pre-empt undetected compromise.

The architecture does not prevent adversarial gaming of governance metadata. An operator authoring calibration claims, drift criteria, or acceptance bands has structural visibility over those declarations, and a sufficiently strategic operator may author declarations that satisfy the substrate's structural commitments while underspecifying the substantive constraints those declarations are intended to express. A calibration claim that is loose enough to admit any observed behaviour produces no drift; a drift criterion whose acceptance band is wide enough to admit substantive failure produces no invalidation. The architecture makes the claim, the criterion, and the resulting compositions structurally visible and attributable to the operator that authored them; it does not detect that the declarations are strategically chosen to evade their nominal purpose. The defence is that the declarations are on the ledger under the operator's identity, available to cross-operator audit and to backtest by counterparties under the cooperative substrate; the substantive judgement that a declaration is strategically permissive is the work of the auditing operator or of the institutional process the cooperative substrate operationalises.

The architecture does not eliminate refusal's operational and human cost. A substrate that refuses structurally produces friction the operator absorbs and, in domains where the downstream of refusal is a person waiting (welfare adjudication, healthcare, emergency response), shifts cost onto the people the system serves. The architecture makes refusal visible and attributable so the operator's refusal-handling policy can be designed and operated as a first-class concern; it does not author the refusal-handling policy, and a substrate whose operator does not author refusal-handling produces refusals that accumulate without recourse. The architectural commitment is that the refusal is structural and traceable, not that refusal is welfare-improving. Whether a refusal-heavy substrate produces net welfare gain in any given domain is the operator's question, mediated by the operator's institutional process and the political authority that authorises that process.

The architecture has centralising as well as plural effects under adoption. The substrate admits multiple sovereigns and the compositional vocabulary is uniform across scales; institutional pluralism is the architectural commitment. The standardisation of governance representation that this requires, however, has historically produced centralising pressure independent of the architecture's design intent: machine-readable authority structures advantage operators with the technical capacity to author them, and substrates whose conforming implementations are interoperable converge on the implementations and policy languages of the largest contributors. The architecture's response is that the architectural specification is independent of any particular implementation and that conforming implementations interoperate at the level of compiled forms rather than at the level of code; whether this is sufficient to preserve plural adoption in practice is an adoption-time question the architecture does not resolve. The monograph develops the adoption treatment; the companion notes the tension explicitly because it bears on how the architecture's defensive properties scale.

The architecture does not interoperate at the wire level until standards-track work is undertaken. The substrate composes established standards (RFC 9162 for witnessing, RFC 5280 for credential structure, W3C Verifiable Credentials for credential expression, Sigstore for transparency logs) but does not itself specify a wire protocol. Conforming implementations interoperate at the level of compiled forms, ledger acts, and cooperative-substrate witnessing; wire-protocol specification is downstream work for IETF or W3C standards bodies, with the architectural specification stabilised first.

The reference implementation does not yet substantiate six of the architecture's commitments at production engineering depth (§4.2): extraction-maturity level as structured unit content; observability provenance as structured unit content; visibility class for policy content; twinning of constitutional source credentials with guardian-quorum revocation; persistent drift state surviving prototype restart; calibration-handling contract as first-class field on the compiled form. The architectural commitments are stable; what differs is engineering depth.

The architecture is not blockchain. Witnessed acts and cryptographic integrity guarantees are shared properties; the operational characteristics differ substantially. The substrate uses content-addressable storage with multi-operator quorum witnessing modelled on Certificate Transparency rather than the consensus mechanisms of distributed ledgers. The substrate's witnessing is multi-custodian and quorum-based; the substrate's archive substrates admit four hosting patterns of which federation is one; the substrate's compiled forms are content-addressed but not chained into a global sequence; the substrate's operational cost and latency are bounded by the witnessing infrastructure, not by a consensus protocol's tractability at scale. The architectural similarity is structural integrity; the architectural difference is what produces it.

The architecture is not policy-as-code. Policy-as-code expresses policy declarations as machine-evaluable artefacts; the substrate's policies are credential units operating in governance role, which is one specialisation of the policy-as-code pattern. The substrate's contribution is the composition discipline above the policy expression: policies are bound through credential graphs, rolled up under strictest-binding-wins, evaluated against compiled forms, invalidated through a uniform surface. The policy expression itself can use whatever policy language a conforming implementation chooses; existing engines (Open Policy Agent, AWS Cedar) are structurally adjacent.

The architecture is not capability-based access control alone. Capability-based systems share lineage with the substrate's credential units, and the substrate's transfer disciplines (bearer, delegated, capability) include the capability pattern as one of three. The substrate's contribution is the composition of capabilities with authority chains, policy roll-up, and compiled-form evaluation at unit commit time; the capability pattern is a primitive within the substrate, not the substrate itself.

The architecture's bounded claim is precise: it makes governance executable, authority attributable, policy composition explicit, refusal structural, and auditability operationally independent of the operator's cooperation in the cases the cooperative substrate hosting admits. Governance itself remains the work of constitutional process, institutional judgement, profes-

sional practice, and political deliberation. The substrate operationalises what these processes produce.

7. Closing

The architecture is three primitive types composing under eight mechanisms; the reference implementation exists, runs against twelve consequential cases, and exposes its specification gaps honestly. The compositional vocabulary is uniform: the same primitives and mechanisms describe a single functional unit on a personal substrate, an institutional substrate composing many units under cooperative-substrate witnessing, and a sovereign substrate operating at multi-national scale.

The monograph develops the constitutional argument for why the mechanism set is the one it is, and why the structural fault the architecture addresses warrants the engineering and institutional investment the architecture's adoption requires. This document specifies what the architecture is at the depth required for conforming implementation. Both engagements are intended.